

SKILL 6 (2520080013)

Database Commands

- **CREATE DATABASE** - Creates a new database.
- **USE** - Selects the database.
- **CREATE TABLE** - Creates a table.
- **INSERT INTO** - Adds records to a table.
- **SELECT** - Displays/retrieves data.
- **UPDATE** - Modifies existing data.
- **DELETE** - Removes records.

```
mysql> CREATE DATABASE BankDB2;  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> USE BankDB2;  
Database changed  
mysql> CREATE TABLE Customer (  
->     Customer_ID INT PRIMARY KEY,  
->     Customer_Name VARCHAR(100) NOT NULL,  
->     Phone VARCHAR(15),  
->     Email VARCHAR(100),  
->     City VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> CREATE TABLE Account (  
->     Account_No INT PRIMARY KEY,  
->     Customer_ID INT,  
->     Account_Type VARCHAR(20),  
->     Balance DECIMAL(12,2) DEFAULT 0,  
->     Branch VARCHAR(50),  
->     FOREIGN KEY (Customer_ID)  
->     REFERENCES Customer(Customer_ID)  
-> );
```

```
mysql> CREATE TABLE Bank_Transaction (  
->     Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  
->     Account_No INT,  
->     Transaction_Type VARCHAR(20),  
->     Amount DECIMAL(12,2),  
->     Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,  
->     FOREIGN KEY (Account_No)  
->     REFERENCES Account(Account_No)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> CREATE TABLE Loan (  
->     Loan_ID INT PRIMARY KEY,  
->     Customer_ID INT,  
->     Loan_Type VARCHAR(30),  
->     Loan_Amount DECIMAL(12,2),  
->     Interest_Rate DECIMAL(5,2),  
->     FOREIGN KEY (Customer_ID)  
->     REFERENCES Customer(Customer_ID)  
-> );  
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> INSERT INTO Customer
-> (Customer_ID, Customer_Name, Phone, Email, City)
-> VALUES
-> (101, 'Ravi Kumar', '9876543210', 'ravi@gmail.com', 'Hyderabad'),
-> (102, 'Priya Sharma', '9876543211', 'priya@gmail.com', 'Vijayawada'),
-> (103, 'Arjun Reddy', '9876543212', 'arjun@gmail.com', 'Bangalore'),
-> (104, 'Sneha Rao', '9876543213', 'sneha@gmail.com', 'Chennai'),
-> (105, 'Kiran Kumar', '9876543214', 'kiran@gmail.com', 'Hyderabad'),
-> (106, 'Anjali Singh', '9876543215', 'anjali@gmail.com', 'Mumbai'),
-> (107, 'Rahul Verma', '9876543216', 'rahul@gmail.com', 'Delhi'),
-> (108, 'Neha Reddy', '9876543217', 'neha@gmail.com', 'Hyderabad'),
-> (109, 'Vikram Rao', '9876543218', 'vikram@gmail.com', 'Pune'),
-> (110, 'Pooja Sharma', '9876543219', 'pooja@gmail.com', 'Chennai');
Query OK, 10 rows affected (0.02 sec)
Records: 10 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO Account
-> (Account_No, Customer_ID, Account_Type, Balance, Branch)
-> VALUES
-> (10001, 101, 'Savings', 50000, 'Hyderabad'),
-> (10002, 102, 'Savings', 75000, 'Vijayawada'),
-> (10003, 103, 'Current', 120000, 'Bangalore'),
-> (10004, 104, 'Savings', 45000, 'Chennai'),
-> (10005, 105, 'Current', 90000, 'Hyderabad'),
-> (10006, 106, 'Savings', 65000, 'Mumbai'),
-> (10007, 107, 'Current', 150000, 'Delhi'),
-> (10008, 108, 'Savings', 55000, 'Hyderabad'),
-> (10009, 109, 'Savings', 85000, 'Pune'),
-> (10010, 110, 'Current', 110000, 'Chennai');
Query OK, 10 rows affected (0.02 sec)
Records: 10 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO Bank_Transaction
-> (Account_No, Transaction_Type, Amount)
-> VALUES
-> (10001, 'DEPOSIT', 10000),
-> (10002, 'DEPOSIT', 15000),
-> (10003, 'WITHDRAW', 20000),
-> (10004, 'DEPOSIT', 5000),
-> (10005, 'WITHDRAW', 10000),
-> (10006, 'DEPOSIT', 8000),
-> (10007, 'WITHDRAW', 15000),
-> (10008, 'DEPOSIT', 12000),
-> (10009, 'DEPOSIT', 7000),
-> (10010, 'WITHDRAW', 5000),
-> (10001, 'DEPOSIT', 3000),
-> (10002, 'WITHDRAW', 5000),
-> (10003, 'DEPOSIT', 10000),
-> (10004, 'WITHDRAW', 3000),
-> (10005, 'DEPOSIT', 6000),
-> (10006, 'WITHDRAW', 4000),
-> (10007, 'DEPOSIT', 20000),
-> (10008, 'WITHDRAW', 5000),
-> (10009, 'DEPOSIT', 9000),
-> (10010, 'DEPOSIT', 11000);
```

Query OK, 20 rows affected (0.01 sec)

Records: 20 Duplicates: 0 Warnings: 0

```
mysql> INSERT INTO Bank_Transaction
-> (Account_No, Transaction_Type, Amount)
-> VALUES
-> (10001, 'DEPOSIT', 10000),
-> (10002, 'DEPOSIT', 15000),
-> (10003, 'WITHDRAW', 20000),
-> (10004, 'DEPOSIT', 5000),
-> (10005, 'WITHDRAW', 10000),
-> (10006, 'DEPOSIT', 8000),
-> (10007, 'WITHDRAW', 15000),
-> (10008, 'DEPOSIT', 12000),
-> (10009, 'DEPOSIT', 7000),
-> (10010, 'WITHDRAW', 5000),
-> (10001, 'DEPOSIT', 3000),
-> (10002, 'WITHDRAW', 5000),
-> (10003, 'DEPOSIT', 10000),
-> (10004, 'WITHDRAW', 3000),
-> (10005, 'DEPOSIT', 6000),
-> (10006, 'WITHDRAW', 4000),
-> (10007, 'DEPOSIT', 20000),
-> (10008, 'WITHDRAW', 5000),
-> (10009, 'DEPOSIT', 9000),
-> (10010, 'DEPOSIT', 11000);
```

```
Query OK, 20 rows affected (0.01 sec)
Records: 20  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO Loan
-> (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)
-> VALUES
-> (501, 101, 'Home Loan', 5000000, 7.5),
-> (502, 102, 'Education Loan', 1000000, 6.5),
-> (503, 103, 'Car Loan', 800000, 8.2),
-> (504, 104, 'Personal Loan', 500000, 10.5),
-> (505, 105, 'Home Loan', 3000000, 7.2),
-> (506, 106, 'Car Loan', 700000, 8.0);
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Customer;
```

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
106	Anjali Singh	9876543215	anjali@gmail.com	Mumbai
107	Rahul Verma	9876543216	rahul@gmail.com	Delhi
108	Neha Reddy	9876543217	neha@gmail.com	Hyderabad
109	Vikram Rao	9876543218	vikram@gmail.com	Pune
110	Pooja Sharma	9876543219	pooja@gmail.com	Chennai

```
10 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Account;
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad
10006	106	Savings	65000.00	Mumbai
10007	107	Current	150000.00	Delhi
10008	108	Savings	55000.00	Hyderabad
10009	109	Savings	85000.00	Pune
10010	110	Current	110000.00	Chennai

```
10 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Bank_Transaction;
```

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
1	10001	DEPOSIT	10000.00	2026-08-12 08:56:43
2	10002	DEPOSIT	15000.00	2026-08-12 08:56:43
3	10003	WITHDRAW	20000.00	2026-08-12 08:56:43
4	10004	DEPOSIT	5000.00	2026-08-12 08:56:43
5	10005	WITHDRAW	10000.00	2026-08-12 08:56:43
6	10006	DEPOSIT	8000.00	2026-08-12 08:56:43
7	10007	WITHDRAW	15000.00	2026-08-12 08:56:43
8	10008	DEPOSIT	12000.00	2026-08-12 08:56:43
9	10009	DEPOSIT	7000.00	2026-08-12 08:56:43
10	10010	WITHDRAW	5000.00	2026-08-12 08:56:43
11	10001	DEPOSIT	3000.00	2026-08-12 08:56:43
12	10002	WITHDRAW	5000.00	2026-08-12 08:56:43
13	10003	DEPOSIT	10000.00	2026-08-12 08:56:43
14	10004	WITHDRAW	3000.00	2026-08-12 08:56:43
15	10005	DEPOSIT	6000.00	2026-08-12 08:56:43
16	10006	WITHDRAW	4000.00	2026-08-12 08:56:43
17	10007	DEPOSIT	20000.00	2026-08-12 08:56:43
18	10008	WITHDRAW	5000.00	2026-08-12 08:56:43
19	10009	DEPOSIT	9000.00	2026-08-12 08:56:43
20	10010	DEPOSIT	11000.00	2026-08-12 08:56:43

```
20 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Loan;
```

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate
501	101	Home Loan	5000000.00	7.50
502	102	Education Loan	1000000.00	6.50
503	103	Car Loan	800000.00	8.20
504	104	Personal Loan	500000.00	10.50
505	105	Home Loan	3000000.00	7.20
506	106	Car Loan	700000.00	8.00

```
6 rows in set (0.00 sec)
```

```
mysql> DELIMITER //
```

```
mysql>
```

```
mysql> CREATE PROCEDURE GetAllCustomers()
```

```
-> BEGIN
```

```
->     SELECT * FROM Customer;
```

```
-> END //
```

```
Query OK, 0 rows affected (0.01 sec)
```

```
mysql>
```

```
mysql> DELIMITER ;
```

```
mysql> DELIMITER //
```

```
mysql>
```

```
mysql> CREATE PROCEDURE GetAllCustomers()
```

```
-> BEGIN
```

```
->     SELECT * FROM Customer;
```

```
-> END //
```

```
ERROR 1304 (42000): PROCEDURE GetAllCustomers already exists
```

```
mysql>
```

```
mysql> DELIMITER ;
```

```
mysql> DELIMITER //
```

A stored procedure is a reusable set of SQL statements stored in the database.

- **CREATE PROCEDURE** - Creates a procedure.
- **CALL** - Executes a procedure.
- **IN** - Accepts input from the user.
- **OUT** - Returns a value from the procedure.
- **DROP PROCEDURE** - Deletes a procedure.

```
mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetAccountDetails(
  ->     IN p_Account_No INT
  -> )
  -> BEGIN
  ->     SELECT *
  ->     FROM Account
  ->     WHERE Account_No = p_Account_No;
  -> END //
```

Query OK, 0 rows affected (0.01 sec)

```
mysql>
mysql> DELIMITER ;
mysql> CALL GetAccountDetails(10001);
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad

```
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetCustomerAccounts(
  ->     IN p_Customer_ID INT
  -> )
  -> BEGIN
  ->     SELECT
  ->         C.Customer_ID,
  ->         C.Customer_Name,
  ->         A.Account_No,
  ->         A.Account_Type,
  ->         A.Balance,
  ->         A.Branch
  ->     FROM Customer C
  ->     JOIN Account A
  ->     ON C.Customer_ID = A.Customer_ID
  ->     WHERE C.Customer_ID = p_Customer_ID;
  -> END //
```

Query OK, 0 rows affected (0.01 sec)

```
mysql>
mysql> DELIMITER ;
mysql> CALL GetCustomerAccounts(101);
```

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Ravi Kumar	10001	Savings	50000.00	Hyderabad

```
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE DepositMoney(
->     IN p_Account_No INT,
->     IN p_Amount DECIMAL(12,2)
-> )
-> BEGIN
->     UPDATE Account
->     SET Balance = Balance + p_Amount
->     WHERE Account_No = p_Account_No;
-> END //
```

Query OK, 0 rows affected (0.01 sec)

```
mysql>
mysql> DELIMITER ;
mysql> CALL DepositMoney(10001, 5000);
Query OK, 1 row affected (0.02 sec)
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE WithdrawMoney(
->     IN p_Account_No INT,
->     IN p_Amount DECIMAL(12,2)
-> )
-> BEGIN
->     UPDATE Account
->     SET Balance = Balance - p_Amount
->     WHERE Account_No = p_Account_No;
-> END //
```

Query OK, 0 rows affected (0.03 sec)


```

mysql>
mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE WithdrawMoney(
    ->     IN p_Account_No INT,
    ->     IN p_Amount DECIMAL(12,2)
    -> )
    -> BEGIN
    ->     UPDATE Account
    ->     SET Balance = Balance - p_Amount
    ->     WHERE Account_No = p_Account_No;
    -> END //
ERROR 1304 (42000): PROCEDURE WithdrawMoney already exists
mysql>
mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER CheckBalance
    -> BEFORE UPDATE ON Account
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.Balance < 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT =
    ->         'Transaction failed: Insufficient balance';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> UPDATE Account
    -> SET Balance = Balance - 60000
    -> WHERE Account_No = 10001;
ERROR 1644 (45000): Transaction failed: Insufficient balance

```

A **trigger** is automatically executed when a specified database event occurs.

- **CREATE TRIGGER** - Creates a trigger.
- **BEFORE INSERT** - Executes before inserting data.
- **BEFORE UPDATE** - Executes before updating data.
- **AFTER INSERT** - Executes after inserting data.
- **DROP TRIGGER** - Deletes a trigger.

Uses: prevent negative balance, validate transaction amounts, maintain transaction audit, and automatically update account balance.

```
mysql>
mysql> DELIMITER ;
mysql> UPDATE Account
  -> SET Balance = Balance - 60000
  -> WHERE Account_No = 10001;
ERROR 1644 (45000): Transaction failed: Insufficient balance
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER CheckTransactionAmount
  -> BEFORE INSERT ON Bank_Transaction
  -> FOR EACH ROW
  -> BEGIN
  ->     IF NEW.Amount <= 0 THEN
  ->         SIGNAL SQLSTATE '45000'
  ->         SET MESSAGE_TEXT =
  ->         'Transaction amount must be greater than zero';
  ->     END IF;
  -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO Bank_Transaction
  -> (Account_No, Transaction_Type, Amount)
  -> VALUES
  -> (10001, 'DEPOSIT', -5000);
ERROR 1644 (45000): Transaction amount must be greater than zero
mysql> CREATE TABLE Transaction_Audit (
  ->     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
  ->     Transaction_ID INT,
  ->     Account_No INT,
  ->     Transaction_Type VARCHAR(20),
  ->     Amount DECIMAL(12,2),
  ->     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
  -> );
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> DELIMITER //
```

```
mysql>
```

```
mysql> CREATE TRIGGER TransactionAudit
```

```
  -> AFTER INSERT ON Bank_Transaction
```

```
  -> FOR EACH ROW
```

```
  -> BEGIN
```

```
  ->     INSERT INTO Transaction_Audit
```

```
  ->     (
```

```
  ->         Transaction_ID,
```

```
  ->         Account_No,
```

```
  ->         Transaction_Type,
```

```
  ->         Amount
```

```
  ->     )
```

```
  ->     VALUES
```

```
  ->     (
```

```
  ->         NEW.Transaction_ID,
```

```
  ->         NEW.Account_No,
```

```
  ->         NEW.Transaction_Type,
```

```
  ->         NEW.Amount
```

```
  ->     );
```

```
  -> END //
```

```
Query OK, 0 rows affected (0.03 sec)
```



```
mysql>
```

```
mysql> DELIMITER ;
```

```
mysql> INSERT INTO Bank_Transaction
```

```
  -> (Account_No, Transaction_Type, Amount)
```

```
  -> VALUES
```

```
  -> (10001, 'DEPOSIT', 2500);
```

```
Query OK, 1 row affected (0.02 sec)
```

```
mysql> DELIMITER ;
mysql> INSERT INTO Bank_Transaction
  -> (Account_No, Transaction_Type, Amount)
  -> VALUES
  -> (10001, 'DEPOSIT', 2500);
Query OK, 1 row affected (0.02 sec)

mysql> SELECT * FROM Transaction_Audit;
+-----+-----+-----+-----+-----+-----+
| Audit_ID | Transaction_ID | Account_No | Transaction_Type | Amount | Audit_Date |
+-----+-----+-----+-----+-----+-----+
| 1 | 21 | 10001 | DEPOSIT | 2500.00 | 2026-08-12 09:00:48 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER UpdateBalanceAfterTransaction
  -> AFTER INSERT ON Bank_Transaction
  -> FOR EACH ROW
  -> BEGIN
  ->     IF NEW.Transaction_Type = 'DEPOSIT' THEN
  ->
  ->         UPDATE Account
  ->         SET Balance = Balance + NEW.Amount
  ->         WHERE Account_No = NEW.Account_No;
  ->
  ->     ELSEIF NEW.Transaction_Type = 'WITHDRAW' THEN
  ->
  ->         UPDATE Account
  ->         SET Balance = Balance - NEW.Amount
  ->         WHERE Account_No = NEW.Account_No;
  ->
  ->     END IF;
  -> END //
```

```
mysql> DELIMITER ;
mysql> INSERT INTO Bank_Transaction
  -> (Account_No, Transaction_Type, Amount)
  -> VALUES
  -> (10001, 'DEPOSIT', 5000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Bank_Transaction
  -> (Account_No, Transaction_Type, Amount)
  -> VALUES
  -> (10001, 'DEPOSIT', 5000);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT *
  -> FROM Account
  -> WHERE Account_No = 10001;
+-----+-----+-----+-----+-----+
| Account_No | Customer_ID | Account_Type | Balance | Branch |
+-----+-----+-----+-----+-----+
| 10001 | 101 | Savings | 65000.00 | Hyderabad |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

```

mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER PreventInsufficientWithdrawal
-> BEFORE INSERT ON Bank_Transaction
-> FOR EACH ROW
-> BEGIN
->     DECLARE CurrentBalance DECIMAL(12,2);
->
->     SELECT Balance
->     INTO CurrentBalance
->     FROM Account
->     WHERE Account_No = NEW.Account_No;
->
->     IF NEW.Transaction_Type = 'WITHDRAW'
->        AND NEW.Amount > CurrentBalance THEN
->
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT =
->         'Withdrawal failed: Insufficient balance';
->
->     END IF;
-> END //
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO Bank_Transaction
-> (Account_No, Transaction_Type, Amount)
-> VALUES
-> (10001, 'WITHDRAW', 1000000);
ERROR 1644 (45000): Withdrawal failed: Insufficient balance
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE TransferMoney(
->     IN SenderAccount INT,
->     IN ReceiverAccount INT,
->     IN TransferAmount DECIMAL(12,2)

```

```
-> DECLARE ReceiverExists INT;
->
-> SELECT COUNT(*)
-> INTO SenderExists
-> FROM Account
-> WHERE Account_No = SenderAccount;
->
-> SELECT COUNT(*)
-> INTO ReceiverExists
-> FROM Account
-> WHERE Account_No = ReceiverAccount;
->
-> IF SenderExists = 0 THEN
->
->     SIGNAL SQLSTATE '45000'
->     SET MESSAGE_TEXT = 'Sender account does not exist';
->
-> ELSEIF ReceiverExists = 0 THEN
->
->     SIGNAL SQLSTATE '45000'
->     SET MESSAGE_TEXT = 'Receiver account does not exist';
->
-> ELSE
->
->     SELECT Balance
->     INTO SenderBalance
->     FROM Account
->     WHERE Account_No = SenderAccount;
->
->     IF TransferAmount <= 0 THEN
->
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Transfer amount must be greater than zero';
->
->     ELSEIF SenderBalance < TransferAmount THEN
->
->         SIGNAL SQLSTATE '45000'
```

```

-> END //
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL TransferMoney(10001, 10002, 5000);
Query OK, 1 row affected (0.01 sec)

```

```

mysql> SELECT *
-> FROM Account
-> WHERE Account_No IN (10001,10002);

```

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	60000.00	Hyderabad
10002	102	Savings	80000.00	Vijayawada

2 rows in set (0.02 sec)

```

mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetCustomerLoans(
->     IN p_Customer_ID INT
-> )
-> BEGIN
->     SELECT
->         C.Customer_Name,
->         L.Loan_ID,
->         L.Loan_Type,
->         L.Loan_Amount,
->         L.Interest_Rate
->     FROM Customer C
->     JOIN Loan L

```

```

mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetCustomerLoans(
->     IN p_Customer_ID INT
-> )
-> BEGIN
->     SELECT
->         C.Customer_Name,
->         L.Loan_ID,
->         L.Loan_Type,
->         L.Loan_Amount,
->         L.Interest_Rate
->     FROM Customer C
->     JOIN Loan L
->     ON C.Customer_ID = L.Customer_ID
->     WHERE C.Customer_ID = p_Customer_ID;
-> END //

```

Query OK, 0 rows affected (0.02 sec)

```

mysql>
mysql> DELIMITER ;
mysql> CALL GetCustomerLoans(101);

```

Customer_Name	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
Ravi Kumar	501	Home Loan	5000000.00	7.50

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE HighBalanceAccounts(
  ->     IN MinimumBalance DECIMAL(12,2)
  -> )
  -> BEGIN
  ->     SELECT *
  ->     FROM Account
  ->     WHERE Balance >= MinimumBalance
  ->     ORDER BY Balance DESC;
  -> END //
```

Query OK, 0 rows affected (0.02 sec)

```
mysql>
mysql> DELIMITER ;
mysql> CALL HighBalanceAccounts(50000);
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10007	107	Current	150000.00	Delhi
10003	103	Current	120000.00	Bangalore
10010	110	Current	110000.00	Chennai
10005	105	Current	90000.00	Hyderabad
10009	109	Savings	85000.00	Pune
10002	102	Savings	80000.00	Vijayawada
10006	106	Savings	65000.00	Mumbai
10001	101	Savings	60000.00	Hyderabad
10008	108	Savings	55000.00	Hyderabad

9 rows in set (0.02 sec)

Query OK, 0 rows affected (0.05 sec)


```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetBalance(
    ->     IN p_Account_No INT,
    ->     OUT p_Balance DECIMAL(12,2)
    -> )
    -> BEGIN
    ->     SELECT Balance
    ->     INTO p_Balance
    ->     FROM Account
    ->     WHERE Account_No = p_Account_No;
    -> END //
```

Query OK, 0 rows affected (0.03 sec)

```
mysql>
mysql> DELIMITER ;
mysql> CALL GetBalance(10001, @CurrentBalance);
Query OK, 1 row affected (0.02 sec)
```

```
mysql>
mysql> SELECT @CurrentBalance;
+-----+
| @CurrentBalance |
+-----+
|          60000.00 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetBranchAccounts(
    ->     IN p_Branch VARCHAR(50)
    -> )
    -> BEGIN
    ->     SELECT *
    ->     FROM Account
    ->     WHERE Branch = p_Branch;
    -> END //
```

Query OK, 0 rows affected (0.02 sec)

```
mysql>
mysql> DELIMITER ;
mysql> CALL GetBranchAccounts('Hyderabad');
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	60000.00	Hyderabad
10005	105	Current	90000.00	Hyderabad
10008	108	Savings	55000.00	Hyderabad

3 rows in set (0.02 sec)

Query OK, 0 rows affected (0.03 sec)

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetCustomerAccountDetails(
  ->     IN p_Customer_ID INT
  -> )
  -> BEGIN
  ->     SELECT
  ->         C.Customer_Name,
  ->         C.Phone,
  ->         C.Email,
  ->         A.Account_No,
  ->         A.Account_Type,
  ->         A.Balance
  ->     FROM Customer C
  ->     JOIN Account A
  ->     ON C.Customer_ID = A.Customer_ID
  ->     WHERE C.Customer_ID = p_Customer_ID;
  -> END //
```

Query OK, 0 rows affected (0.02 sec)

```
mysql>
mysql> DELIMITER ;
mysql> CALL GetCustomerAccountDetails(101);
```

Customer_Name	Phone	Email	Account_No	Account_Type	Balance
Ravi Kumar	9876543210	ravi@gmail.com	10001	Savings	60000.00

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetTotalCustomerBalance(
->     IN p_Customer_ID INT
-> )
-> BEGIN
->     SELECT
->         C.Customer_Name,
->         SUM(A.Balance) AS Total_Balance
->     FROM Customer C
->     JOIN Account A
->     ON C.Customer_ID = A.Customer_ID
->     WHERE C.Customer_ID = p_Customer_ID
->     GROUP BY C.Customer_ID, C.Customer_Name;
-> END //
```

Query OK, 0 rows affected (0.02 sec)

```
mysql>
mysql> DELIMITER ;
mysql> CALL GetTotalCustomerBalance(101);
```

Customer_Name	Total_Balance
Ravi Kumar	60000.00

1 row in set (0.02 sec)

Query OK, 0 rows affected (0.02 sec)

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetAccountsAboveBalance(
->     IN p_Amount DECIMAL(12,2)
-> )
-> BEGIN
->     SELECT *
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetAccountsAboveBalance(
->     IN p_Amount DECIMAL(12,2)
-> )
-> BEGIN
->     SELECT *
->     FROM Account
->     WHERE Balance > p_Amount
->     ORDER BY Balance DESC;
-> END //
```

Query OK, 0 rows affected (0.03 sec)

```
mysql>
mysql> DELIMITER ;
mysql> CALL GetAccountsAboveBalance(100000);
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10007	107	Current	150000.00	Delhi
10003	103	Current	120000.00	Bangalore
10010	110	Current	110000.00	Chennai

3 rows in set (0.00 sec)

Query OK, 0 rows affected (0.02 sec)

```

mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER PreventNegativeBalance
-> BEFORE UPDATE ON Account
-> FOR EACH ROW
-> BEGIN
->     IF NEW.Balance < 0 THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Account balance cannot be negative';
->     END IF;
-> END //
Query OK, 0 rows affected (0.01 sec)

```

```

mysql>
mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER PreventInvalidTransaction
-> BEFORE INSERT ON Bank_Transaction
-> FOR EACH ROW
-> BEGIN
->     IF NEW.Amount <= 0 THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT =
->         'Transaction amount must be greater than zero';
->     END IF;
-> END //
Query OK, 0 rows affected (0.03 sec)

```

```

mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER AddTransactionAudit
-> AFTER INSERT ON Bank_Transaction
-> FOR EACH ROW
-> BEGIN
->     INSERT INTO Transaction_Audit
->     (
->         Transaction_ID,
->         Account_No,
->         Transaction_Type,
->         Amount
->     )
->     VALUES
->     (
->         NEW.Transaction_ID,
->         NEW.Account_No,
->         NEW.Transaction_Type,
->         NEW.Amount
->     );
-> END //
Query OK, 0 rows affected (0.01 sec)

```

```

mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER DepositBalanceUpdate
-> AFTER INSERT ON Bank_Transaction
-> FOR EACH ROW
-> BEGIN
->     IF NEW.Transaction_Type = 'DEPOSIT' THEN
->         UPDATE Account
->         SET Balance = Balance + NEW.Amount
->         WHERE Account_No = NEW.Account_No;
->     END IF;
-> END //

```

Query OK, 0 rows affected (0.01 sec)

```

mysql>
mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER WithdrawalBalanceUpdate
-> AFTER INSERT ON Bank_Transaction
-> FOR EACH ROW
-> BEGIN
->     IF NEW.Transaction_Type = 'WITHDRAW' THEN
->         UPDATE Account
->         SET Balance = Balance - NEW.Amount
->         WHERE Account_No = NEW.Account_No;
->     END IF;
-> END //

```

Query OK, 0 rows affected (0.01 sec)

```

mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER CheckWithdrawalBalance
-> BEFORE INSERT ON Bank_Transaction
-> FOR EACH ROW
-> BEGIN
->     DECLARE CurrentBalance DECIMAL(12,2);
->
->     SELECT Balance
->     INTO CurrentBalance
->     FROM Account
->     WHERE Account_No = NEW.Account_No;
->
->     IF NEW.Transaction_Type = 'WITHDRAW'
->     AND NEW.Amount > CurrentBalance THEN
->
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT =
->         'Withdrawal failed: Insufficient balance';
->
->     END IF;
-> END //

```

Query OK, 0 rows affected (0.01 sec)

```
mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE TransferMoneySafe(
->     IN p_Sender INT,
->     IN p_Receiver INT,
->     IN p_Amount DECIMAL(12,2)
-> )
-> BEGIN
->     DECLARE SenderBalance DECIMAL(12,2);
->     DECLARE SenderCount INT;
->     DECLARE ReceiverCount INT;
->
->     SELECT COUNT(*)
->     INTO SenderCount
->     FROM Account
->     WHERE Account_No = p_Sender;
->
->     SELECT COUNT(*)
->     INTO ReceiverCount
->     FROM Account
->     WHERE Account_No = p_Receiver;
->
->     IF SenderCount = 0 THEN
->
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Sender account does not exist';
->
->     ELSEIF ReceiverCount = 0 THEN
->
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Receiver account does not exist';
->
->     ELSEIF p_Amount <= 0 THEN
->
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Amount must be greater than zero';
```

```

MySQL 8.0 Command Line C
-> SET MESSAGE_TEXT = 'Amount must be greater than zero';
->
-> ELSE
->
-> SELECT Balance
-> INTO SenderBalance
-> FROM Account
-> WHERE Account_No = p_Sender;
->
-> IF SenderBalance < p_Amount THEN
->
-> SIGNAL SQLSTATE '45000'
-> SET MESSAGE_TEXT = 'Insufficient balance';
->
-> ELSE
->
-> UPDATE Account
-> SET Balance = Balance - p_Amount
-> WHERE Account_No = p_Sender;
->
-> UPDATE Account
-> SET Balance = Balance + p_Amount
-> WHERE Account_No = p_Receiver;
->
-> END IF;
-> END IF;
-> END //
Query OK, 0 rows affected (0.00 sec)

```

```

mysql> DELIMITER ;
mysql> CALL TransferMoneySafe(10001, 10002, 5000);
Query OK, 1 row affected (0.01 sec)

mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE CalculateLoanInterest(
-> IN p_LoanAmount DECIMAL(12,2),
-> IN p_InterestRate DECIMAL(5,2),
-> IN p_Years INT
-> )
-> BEGIN
-> DECLARE SimpleInterest DECIMAL(12,2);
-> DECLARE TotalAmount DECIMAL(12,2);
->
-> SET SimpleInterest =
-> (p_LoanAmount * p_InterestRate * p_Years) / 100;
->
-> SET TotalAmount =
-> p_LoanAmount + SimpleInterest;
->
-> SELECT
-> SimpleInterest,
-> TotalAmount;
-> END //
Query OK, 0 rows affected (0.01 sec)

```

- **IF / ELSEIF / ELSE** - Performs operations based on conditions.
- **DECLARE** - Creates a local variable.
- **SELECT ... INTO** - Stores a selected value in a variable.
- **SIGNAL SQLSTATE '45000'** - Generates a custom error and stops an invalid operation.
- **DELIMITER** - Allows procedures/triggers containing multiple SQL statements to be defined.

```
mysql> DELIMITER ;
mysql> CALL CalculateLoanInterest(500000, 7.5, 5);
+-----+-----+
| SimpleInterest | TotalAmount |
+-----+-----+
|      187500.00 |    687500.00 |
+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER PreventCustomerDelete
-> BEFORE DELETE ON Customer
-> FOR EACH ROW
-> BEGIN
->     DECLARE AccountCount INT;
->
->     SELECT COUNT(*)
->     INTO AccountCount
->     FROM Account
->     WHERE Customer_ID = OLD.Customer_ID;
->
->     IF AccountCount > 0 THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT =
->         'Customer cannot be deleted because an account exists';
->     END IF;
-> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
mysql> DELETE FROM Customer
-> WHERE Customer_ID = 101;
ERROR 1644 (45000): Customer cannot be deleted because an account exists
mysql> CREATE TABLE Account_Audit (
->     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
->     Account_No INT,
->     Old_Balance DECIMAL(12,2),
->     New_Balance DECIMAL(12,2),
->     Changed_Date DATETIME DEFAULT CURRENT_TIMESTAMP
-> );
Query OK, 0 rows affected (0.03 sec)
```


- **SHOW TRIGGERS** - Displays available triggers.
- **SHOW CREATE TRIGGER** - Displays the code of a trigger.
- **SHOW PROCEDURE STATUS** - Displays stored procedures.
- **DROP TRIGGER / DROP PROCEDURE** - Removes them.

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER AccountBalanceAudit
-> AFTER UPDATE ON Account
-> FOR EACH ROW
-> BEGIN
->     IF OLD.Balance <> NEW.Balance THEN
->
->         INSERT INTO Account_Audit
->         (
->             Account_No,
->             Old_Balance,
->             New_Balance
->         )
->         VALUES
->         (
->             OLD.Account_No,
->             OLD.Balance,
->             NEW.Balance
->         );
->
->     END IF;
-> END //
Query OK, 0 rows affected (0.01 sec)
```

```
mysql>
mysql> DELIMITER ;
mysql> SELECT * FROM Account_Audit;
Empty set (0.00 sec)
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE Top5Accounts()
-> BEGIN
->     SELECT *
->     FROM Account
->     ORDER BY Balance DESC
```

```
mysql>
mysql> DELIMITER ;
mysql> SELECT * FROM Account_Audit;
Empty set (0.00 sec)
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE Top5Accounts()
-> BEGIN
->     SELECT *
->     FROM Account
->     ORDER BY Balance DESC
->     LIMIT 5;
-> END //
```

Query OK, 0 rows affected (0.01 sec)

```
mysql>
mysql> DELIMITER ;
mysql> CALL Top5Accounts();
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10007	107	Current	150000.00	Delhi
10003	103	Current	120000.00	Bangalore
10010	110	Current	110000.00	Chennai
10005	105	Current	90000.00	Hyderabad
10002	102	Savings	85000.00	Vijayawada

5 rows in set (0.02 sec)

Query OK, 0 rows affected (0.04 sec)

```
mysql> CALL Top5Accounts();
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10007	107	Current	150000.00	Delhi
10003	103	Current	120000.00	Bangalore
10010	110	Current	110000.00	Chennai
10005	105	Current	90000.00	Hyderabad
10002	102	Savings	85000.00	Vijayawada

5 rows in set (0.02 sec)

Query OK, 0 rows affected (0.04 sec)

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE CountAccountsByBranch()
-> BEGIN
->     SELECT
->         Branch,
->         COUNT(*) AS Number_of_Accounts
->     FROM Account
->     GROUP BY Branch;
-> END //
```

Query OK, 0 rows affected (0.02 sec)

```
mysql>
mysql> DELIMITER ;
mysql> CALL CountAccountsByBranch();
```

```
mysql> CALL CountAccountsByBranch();
```

Branch	Number_of_Accounts
Hyderabad	3
Vijayawada	1
Bangalore	1
Chennai	2
Mumbai	1
Delhi	1
Pune	1

```
7 rows in set (0.02 sec)
```

```
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> DELIMITER //
```

```
mysql>
```

```
mysql> CREATE PROCEDURE CustomersWithHighLoans(
```

```
    ->     IN p_Amount DECIMAL(12,2)
```

```
    -> )
```

```
    -> BEGIN
```

```
    ->     SELECT
```

```
    ->         C.Customer_ID,
```

```
    ->         C.Customer_Name,
```

```
    ->         L.Loan_ID,
```

```
    ->         L.Loan_Type,
```

```
    ->         L.Loan_Amount,
```

```
    ->         L.Interest_Rate
```

```
    ->     FROM Customer C
```

```
    ->     JOIN Loan L
```

```
    ->     ON C.Customer_ID = L.Customer_ID
```

```
    ->     WHERE L.Loan_Amount > p_Amount;
```

```
    -> END //
```

```
Query OK, 0 rows affected (0.03 sec)
```

```
mysql>
mysql> DELIMITER ;
mysql> CALL CustomersWithHighLoans(1000000);
```

Customer_ID	Customer_Name	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
101	Ravi Kumar	501	Home Loan	5000000.00	7.50
105	Kiran Kumar	505	Home Loan	3000000.00	7.20

```
2 rows in set (0.02 sec)
```

Query OK, 0 rows affected (0.02 sec)

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER PreventAccountDelete
-> BEFORE DELETE ON Account
-> FOR EACH ROW
-> BEGIN
->     DECLARE TransactionCount INT;
->
->     SELECT COUNT(*)
->     INTO TransactionCount
->     FROM Bank_Transaction
->     WHERE Account_No = OLD.Account_No;
->
->     IF TransactionCount > 0 THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT =
->         'Account cannot be deleted because transactions exist';
->     END IF;
-> END //
```

Query OK, 0 rows affected (0.01 sec)

```
mysql>
mysql> DELIMITER ;
mysql> DELETE FROM Account
-> WHERE Account_No = 10001;
ERROR 1644 (45000): Account cannot be deleted because transactions exist
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetTransactionHistory(
->     IN p_Account_No INT
-> )
-> BEGIN
->     SELECT
->         Transaction_ID,
->         Account_No,
->         Transaction_Type,
->         Amount,
->         Transaction_Date
->     FROM Bank_Transaction
->     WHERE Account_No = p_Account_No
->     ORDER BY Transaction_Date DESC;
-> END //
```

Query OK, 0 rows affected (0.00 sec)

```

mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetTransactionHistory(
->     IN p_Account_No INT
-> )
-> BEGIN
->     SELECT
->         Transaction_ID,
->         Account_No,
->         Transaction_Type,
->         Amount,
->         Transaction_Date
->     FROM Bank_Transaction
->     WHERE Account_No = p_Account_No
->     ORDER BY Transaction_Date DESC;
-> END //
ERROR 1304 (42000): PROCEDURE GetTransactionHistory already exists
mysql>
mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetAccountSummary()
-> BEGIN
->     SELECT
->         A.Account_No,
->         C.Customer_Name,
->         A.Account_Type,
->         A.Balance,
->         A.Branch
->     FROM Account A
->     JOIN Customer C
->     ON A.Customer_ID = C.Customer_ID;
-> END //
Query OK, 0 rows affected (0.01 sec)

```

```

mysql>
mysql> DELIMITER ;
mysql> CALL GetAccountSummary();
+-----+-----+-----+-----+-----+
| Account_No | Customer_Name | Account_Type | Balance | Branch |
+-----+-----+-----+-----+-----+
| 10001 | Ravi Kumar | Savings | 55000.00 | Hyderabad |
| 10002 | Priya Sharma | Savings | 85000.00 | Vijayawada |
| 10003 | Arjun Reddy | Current | 120000.00 | Bangalore |
| 10004 | Sneha Rao | Savings | 45000.00 | Chennai |
| 10005 | Kiran Kumar | Current | 90000.00 | Hyderabad |
| 10006 | Anjali Singh | Savings | 65000.00 | Mumbai |
| 10007 | Rahul Verma | Current | 150000.00 | Delhi |
| 10008 | Neha Reddy | Savings | 55000.00 | Hyderabad |
| 10009 | Vikram Rao | Savings | 85000.00 | Pune |
| 10010 | Pooja Sharma | Current | 110000.00 | Chennai |
+-----+-----+-----+-----+-----+
10 rows in set (0.00 sec)

Query OK, 0 rows affected (0.04 sec)

mysql>

```